

Machine Learning ETE Question 5

```
In [16]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay

RANDOM_STATE = 42
```

1. Load the Dataset

```
In [17]: iris = load_iris(as_frame=True)
X, y = iris.data, iris.target

# Map numeric labels to species names for readability
label_map = dict(enumerate(iris.target_names))

print(f'Samples : {X.shape[0]}')
print(f'Features: {X.shape[1]}')
print(f'Classes : {list(iris.target_names)}')
display(X.head())
```

Samples : 150

Features: 4

Classes : [np.str_('setosa'), np.str_('versicolor'), np.str_('virginica')]

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

2. Exploratory Data Analysis (EDA)

2.1 Basic Statistics & Data Quality

```
In [18]: print('=== Descriptive Statistics ===')
display(X.describe().round(2))

print('\n=== Data Types ===')
display(X.dtypes)

print(f'\n=== Missing Values ===')
missing = X.isnull().sum()
print(missing.to_string())
print(f'Total missing: {missing.sum()}')

print(f'\n=== Duplicate Rows ===')
n_dupes = X.duplicated().sum()
print(f'Duplicate rows: {n_dupes}')
```

=== Descriptive Statistics ===

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
count	150.00	150.00	150.00	150.00
mean	5.84	3.06	3.76	1.20
std	0.83	0.44	1.77	0.76
min	4.30	2.00	1.00	0.10
25%	5.10	2.80	1.60	0.30
50%	5.80	3.00	4.35	1.30
75%	6.40	3.30	5.10	1.80
max	7.90	4.40	6.90	2.50

=== Data Types ===

```
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
dtype: object
```

=== Missing Values ===

```
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
Total missing: 0
```

=== Duplicate Rows ===

```
Duplicate rows: 1
```

2.2 Class Distribution

```
In [19]: class_counts = y.value_counts().sort_index().rename(label_map)
print('Class distribution:')
display(class_counts)
```

```
fig, ax = plt.subplots(figsize=(5, 3))
class_counts.plot.bar(color=['#4C72B0', '#55A868', '#C44E52'], edgecolor='black', a
ax.set_ylabel('Count')
ax.set_title('Class Distribution')
ax.set_xticklabels(ax.get_xticklabels(), rotation=0)
plt.tight_layout()
plt.show()
```

Class distribution:

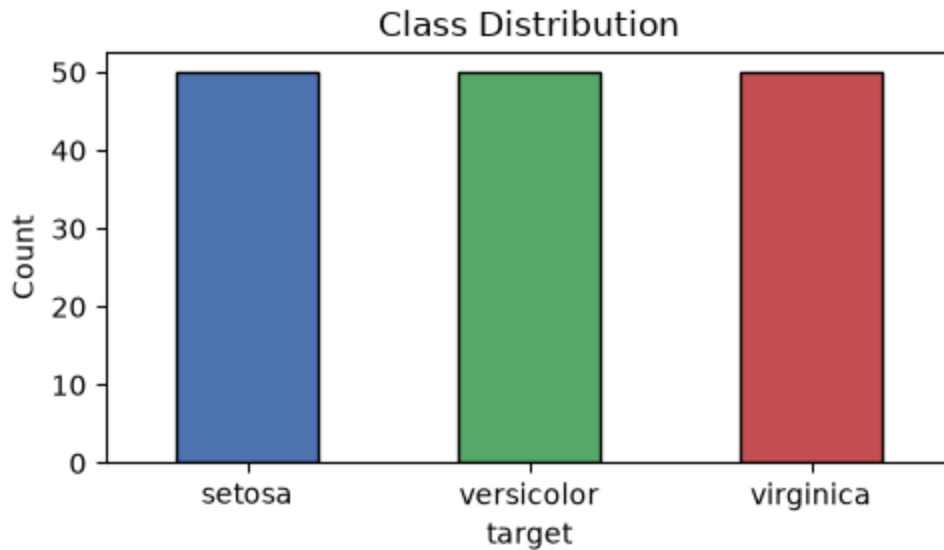
target

setosa 50

versicolor 50

virginica 50

Name: count, dtype: int64

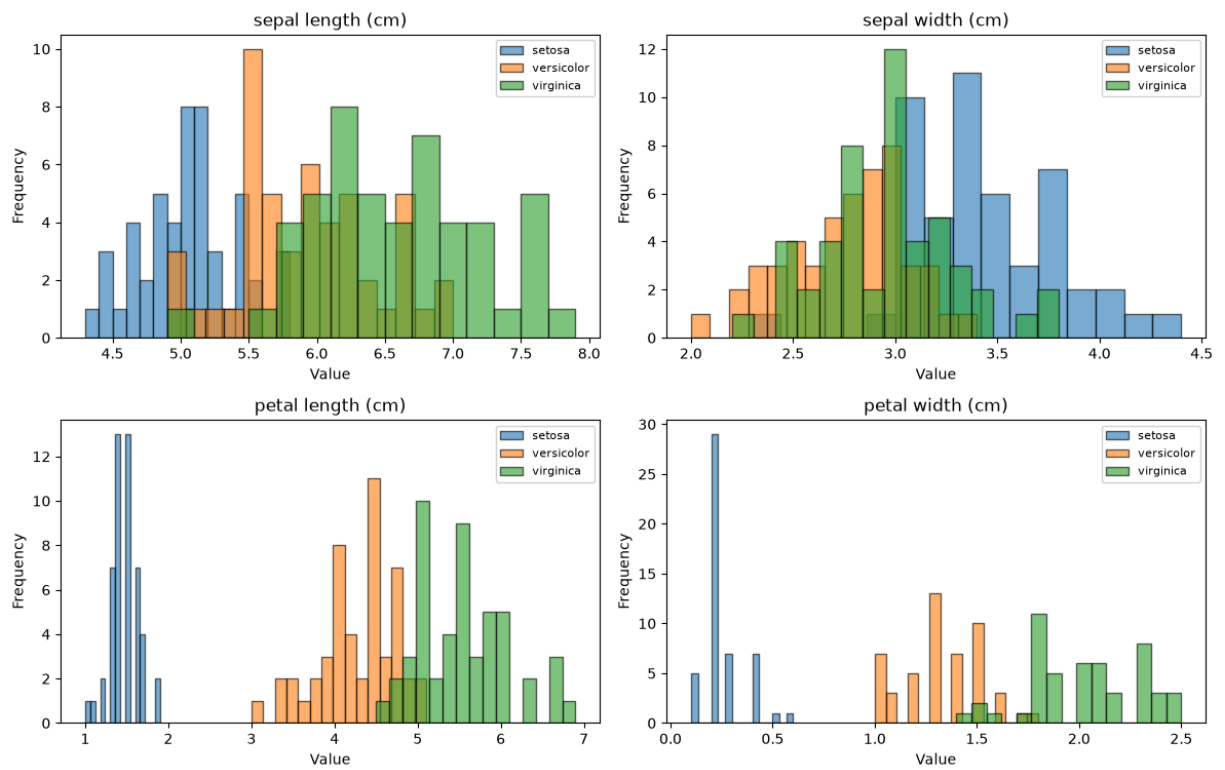


2.3 Feature Distributions by Class

```
In [20]: df_plot = X.copy()
df_plot['species'] = y.map(label_map)

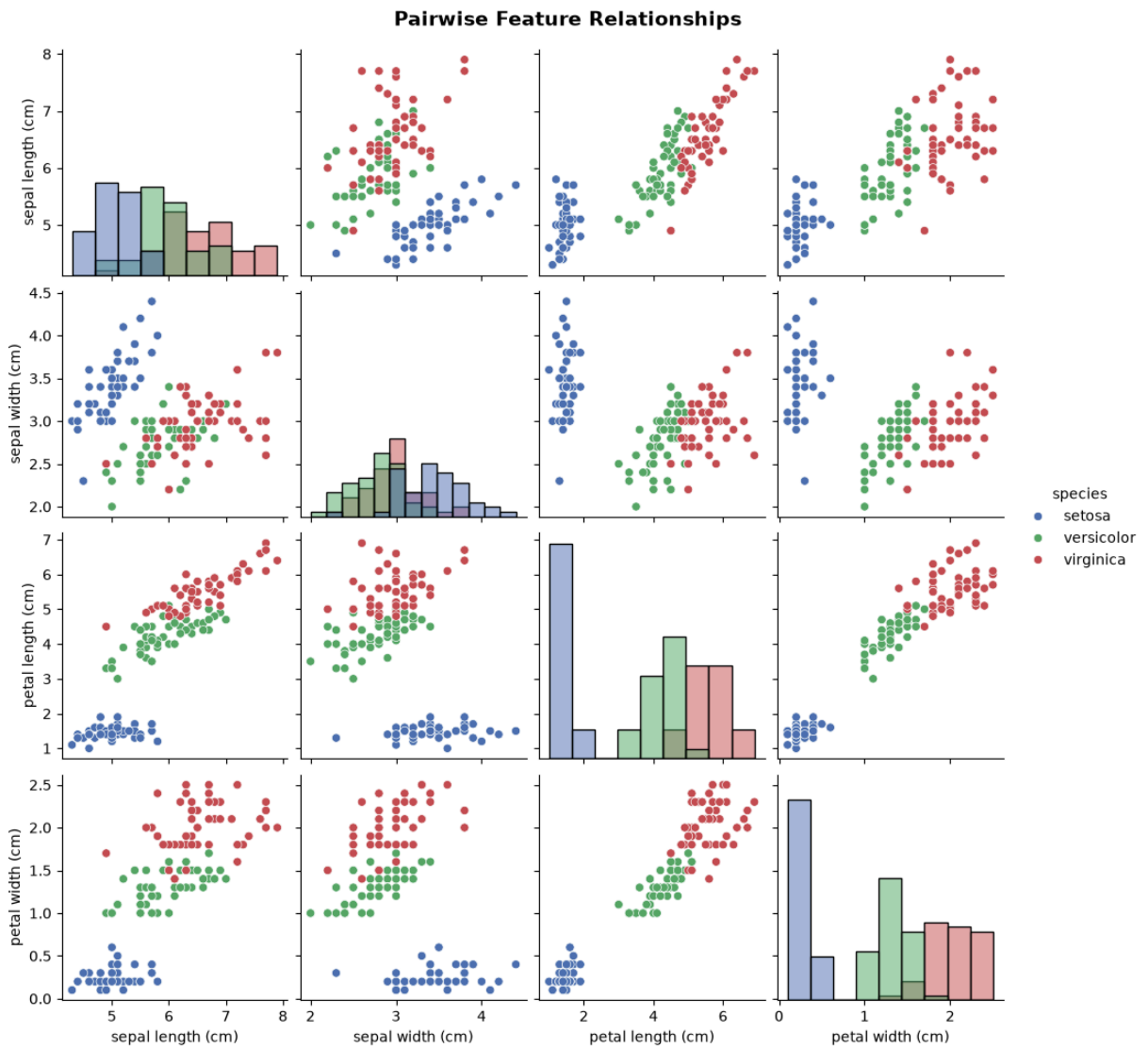
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
for ax, col in zip(axes.ravel(), X.columns):
    for species in iris.target_names:
        subset = df_plot[df_plot['species'] == species][col]
        ax.hist(subset, bins=15, alpha=0.6, label=species, edgecolor='black')
    ax.set_title(col)
    ax.set_xlabel('Value')
    ax.set_ylabel('Frequency')
    ax.legend(fontsize=8)
plt.suptitle('Feature Distributions by Class', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```

Feature Distributions by Class



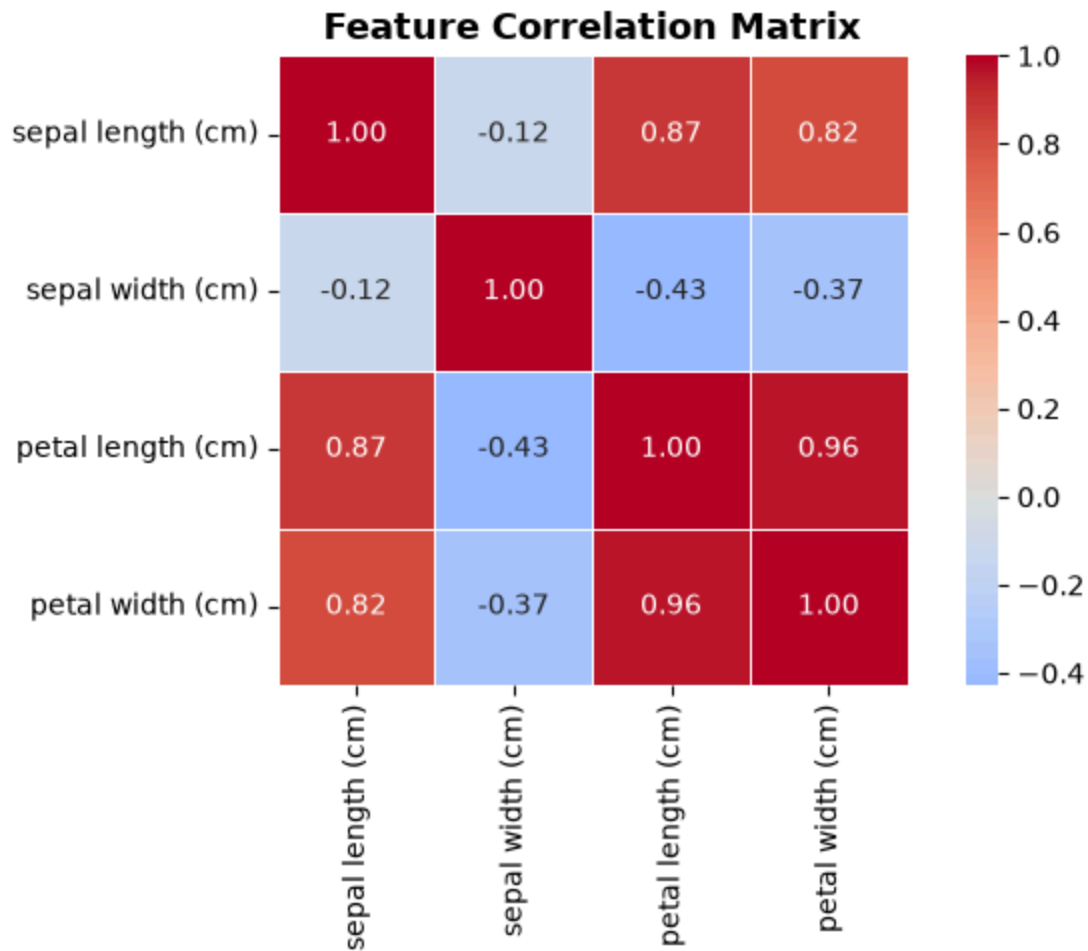
2.4 Pairplot

```
In [21]: sns.pairplot(df_plot, hue='species', diag_kind='hist',
                    palette={'setosa': '#4C72B0', 'versicolor': '#55A868', 'virginica': '#55A868'},
                    plt.suptitle('Pairwise Feature Relationships', y=1.02, fontsize=14, fontweight='bold'),
                    plt.show())
```



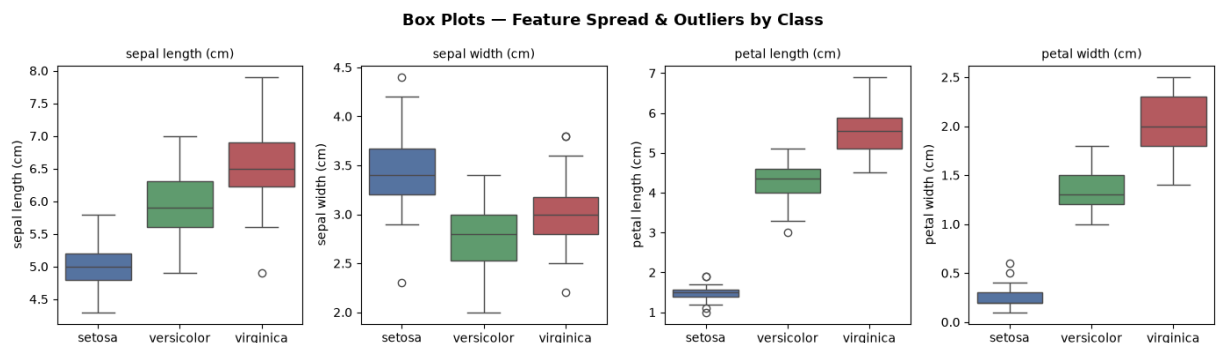
2.5 Correlation Heatmap

```
In [22]: fig, ax = plt.subplots(figsize=(7, 5))
corr = X.corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0,
            square=True, linewidths=0.5, ax=ax)
ax.set_title('Feature Correlation Matrix', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```



2.6 Box Plots (Outlier Detection)

```
In [23]: fig, axes = plt.subplots(1, 4, figsize=(14, 4))
for ax, col in zip(axes, X.columns):
    sns.boxplot(x='species', y=col, data=df_plot, hue='species', ax=ax,
                palette={'setosa': '#4C72B0', 'versicolor': '#55A868', 'virginica':
                        '#E31A1C'}, legend=False)
    ax.set_title(col, fontsize=10)
    ax.set_xlabel('')
plt.suptitle('Box Plots – Feature Spread & Outliers by Class', fontsize=13, fontwei
plt.tight_layout()
plt.show()
```



3. Data Preprocessing

Summary of findings from EDA:

- No missing values — no imputation needed.
- All features are continuous (`float64`) — no encoding required.
- Classes are perfectly balanced (50 each) — no resampling needed.
- A few mild outliers in sepal width, but they are real observations, not data errors.
- Feature scales differ (e.g., sepal length ~4–8 vs petal width ~0–2.5), so **standardisation is essential** for the MLP, which uses gradient-based optimisation.

Standardisation (`StandardScaler`) is applied inside a `Pipeline` to prevent data leakage during cross-validation.

3.1 Train-Test Split

```
In [24]: X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
    )

    print(f'Training samples: {X_train.shape[0]}')
    print(f'Test samples      : {X_test.shape[0]}')
```

Training samples: 120

Test samples : 30

3.2 Feature Scaling (Preview)

The scaler is fitted only on the training data to demonstrate the effect. In the actual model, scaling is handled inside the pipeline.

```
In [25]: scaler_preview = StandardScaler().fit(X_train)
        X_train_scaled = pd.DataFrame(
            scaler_preview.transform(X_train), columns=X.columns, index=X_train.index
        )

        print('Before scaling:')
        display(X_train.describe().round(2).loc[['mean', 'std', 'min', 'max']])

        print('\nAfter scaling (mean ≈ 0, std ≈ 1):')
        display(X_train_scaled.describe().round(2).loc[['mean', 'std', 'min', 'max']])
```

Before scaling:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
mean	5.84	3.05	3.77	1.20
std	0.84	0.45	1.77	0.76
min	4.30	2.00	1.10	0.10
max	7.90	4.40	6.90	2.50

After scaling (mean ≈ 0 , std ≈ 1):

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
mean	-0.00	-0.00	0.00	0.00
std	1.00	1.00	1.00	1.00
min	-1.84	-2.35	-1.52	-1.45
max	2.46	3.03	1.78	1.71

4. Define Two MLP Architectures

Architecture	Hidden Layers	Description
A – Shallow	(50,)	Single hidden layer with 50 neurons
B – Deep	(50, 30)	Two hidden layers with 50 and 30 neurons

Both pipelines standardise features before feeding them to the MLP, since neural networks are sensitive to feature scale.

```
In [26]: architectures = {
    'MLP-A (50,)': (50,),
    'MLP-B (50, 30)': (50, 30),
}

pipelines = {}
for name, hidden in architectures.items():
    pipelines[name] = Pipeline([
        ('scaler', StandardScaler()),
        ('mlp', MLPClassifier(
            hidden_layer_sizes=hidden,
            activation='relu',
            solver='adam',
            max_iter=1000,
            random_state=RANDOM_STATE,
        )),
    ])

print('Pipelines created:')
```



```
for name in pipelines:
    print(f' • {name}')
```

Pipelines created:

- MLP-A (50,)
- MLP-B (50, 30)

5. Cross-Validation Comparison

5-fold stratified cross-validation on the **training set** ensures each fold preserves the class ratio.

```
In [27]: cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

cv_results = []
for name, pipe in pipelines.items():
    scores = cross_val_score(pipe, X_train, y_train, cv=cv, scoring='accuracy')
    cv_results.append({
        'Architecture': name,
        'Mean CV Accuracy': scores.mean(),
        'Std CV Accuracy': scores.std(),
        'Fold Scores': scores,
    })
    print(f'{name}: {scores.mean():.4f} ± {scores.std():.4f} (folds: {np.round(sc

cv_df = pd.DataFrame(cv_results).set_index('Architecture')[['Mean CV Accuracy', 'St
display(cv_df.round(4))
```

```
MLP-A (50,): 0.9583 ± 0.0264 (folds: [0.9583 1. 0.9583 0.9583 0.9167])
MLP-B (50, 30): 0.9500 ± 0.0312 (folds: [0.9583 1. 0.9167 0.9583 0.9167])
```

Mean CV Accuracy Std CV Accuracy

Architecture

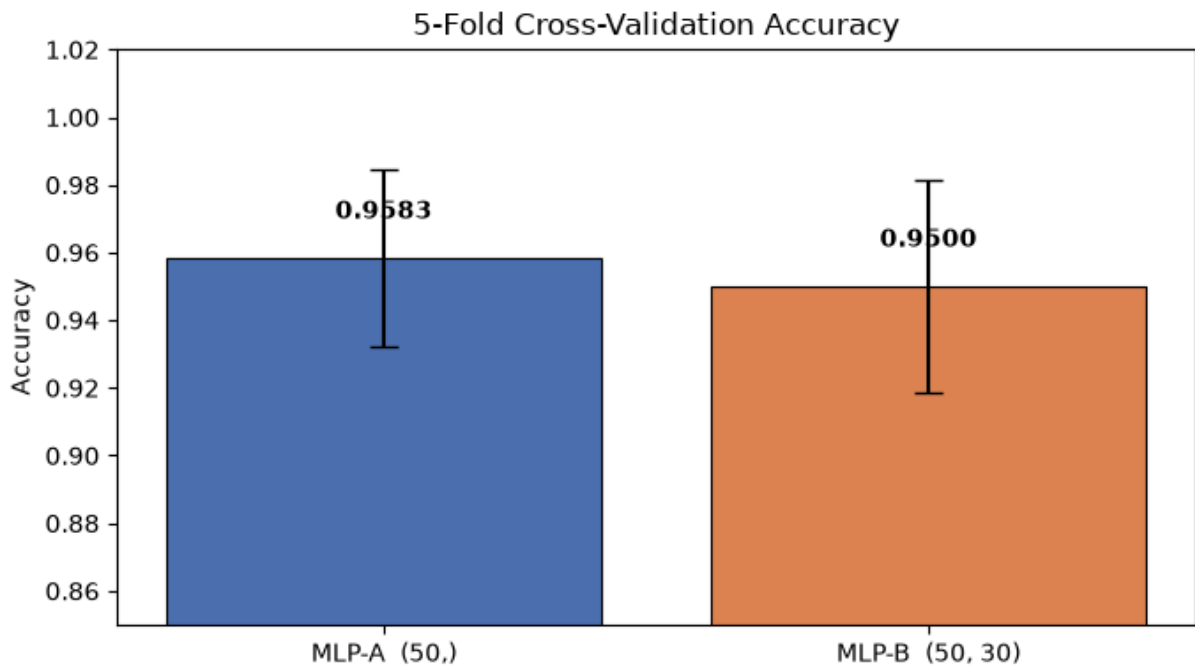
MLP-A (50,)	0.9583	0.0264
MLP-B (50, 30)	0.9500	0.0312

```
In [28]: # Bar chart of cross-validation accuracy
fig, ax = plt.subplots(figsize=(7, 4))
bars = ax.bar(
    cv_df.index,
    cv_df['Mean CV Accuracy'],
    yerr=cv_df['Std CV Accuracy'],
    capsize=6,
    color=['#4C72B0', '#DD8452'],
    edgecolor='black',
    linewidth=0.8,
)
ax.set_ylabel('Accuracy')
ax.set_title('5-Fold Cross-Validation Accuracy')
ax.set_ylim(0.85, 1.02)
for bar, val in zip(bars, cv_df['Mean CV Accuracy']):
    ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.01,
```

```

        f'{val:.4f}', ha='center', va='bottom', fontweight='bold')
plt.tight_layout()
plt.show()

```



6. Train Best Architecture and Evaluate on Test Set

```

In [29]: # Select the architecture with the highest mean CV accuracy
best_name = cv_df['Mean CV Accuracy'].idxmax()
best_pipe = pipelines[best_name]
print(f'Best architecture by CV: {best_name}')

# Fit on the entire training set
best_pipe.fit(X_train, y_train)

# Predict on held-out test set
y_pred = best_pipe.predict(X_test)
test_acc = accuracy_score(y_test, y_pred)
print(f'Test Accuracy: {test_acc:.4f}')

```

Best architecture by CV: MLP-A (50,)
 Test Accuracy: 0.9667

7. Confusion Matrix

```

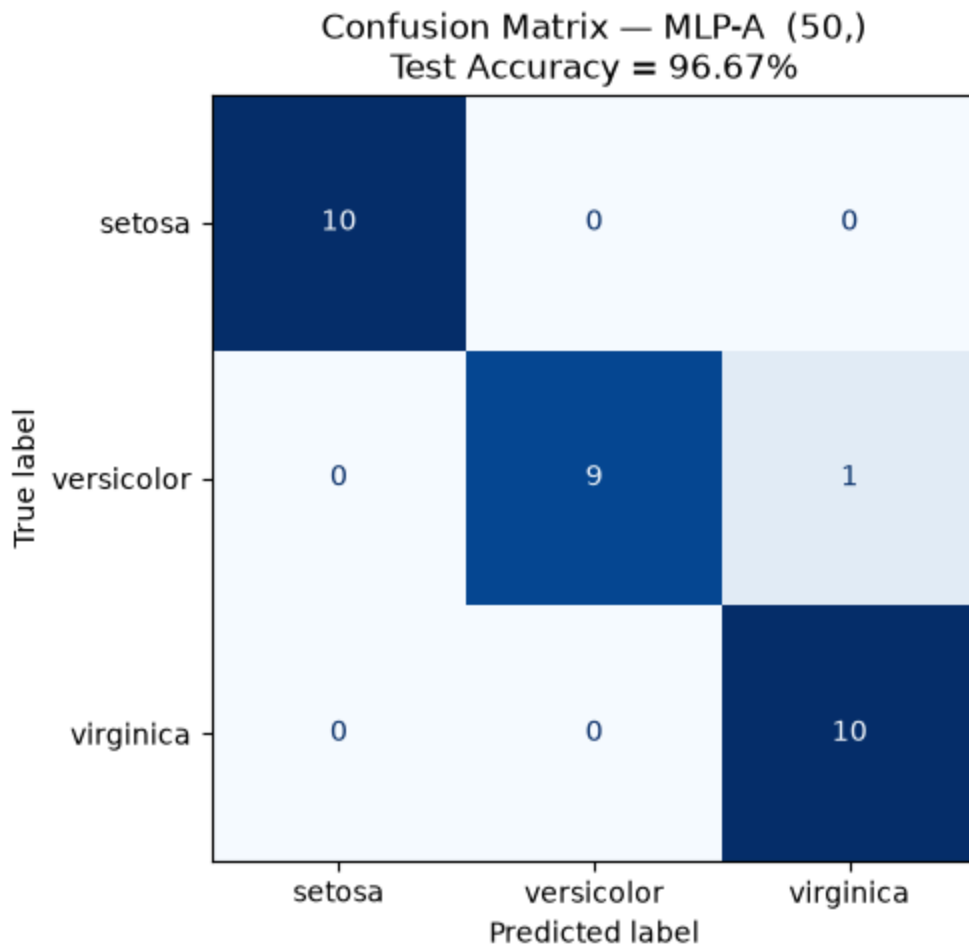
In [30]: cm = confusion_matrix(y_test, y_pred)

fig, ax = plt.subplots(figsize=(6, 5))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=iris.target_names)
disp.plot(cmap='Blues', ax=ax, colorbar=False)
ax.set_title(f'Confusion Matrix - {best_name}\nTest Accuracy = {test_acc:.2%}')
plt.tight_layout()

```

```
plt.show()

print('\nConfusion matrix (raw counts):')
display(pd.DataFrame(cm, index=iris.target_names, columns=iris.target_names))
```



Confusion matrix (raw counts):

	setosa	versicolor	virginica
setosa	10	0	0
versicolor	0	9	1
virginica	0	0	10

Discussion

- **EDA findings:** The dataset is clean — no missing values, no encoding needed, and perfectly balanced classes. Petal features show the strongest class separation, while sepal width has the most overlap. Petal length and petal width are highly correlated ($r \approx 0.96$).
- **Preprocessing:** Feature standardisation is the key preprocessing step, as MLPs use gradient-based optimisation and are sensitive to feature scale. This is handled inside the pipeline to avoid data leakage.

- **Architecture comparison:** Cross-validation shows whether the deeper network (50, 30) offers an advantage over the single-layer (50,) design. On a small, low-dimensional dataset like Iris, both architectures typically achieve high accuracy (~95–100%), so the simpler model is usually preferable.
- **Confusion matrix insights:** Most misclassifications, if any, occur between *versicolor* and *virginica*, which overlap in feature space. *Setosa* is almost always perfectly separated.

Conclusion: For the Iris dataset, a single hidden layer with moderate width is sufficient. Adding depth provides marginal — if any — improvement, and the simpler architecture is easier to train and interpret.